TEAM APEX · BATCH 2_5 · COURSE PROJECT

Semester Planning Agent

An AI agent that plans a Technion semester like an experienced academic advisor — built, tested, and hardened live over several weeks, not just designed once and shipped.

Manhal Ghoummaid · Dima Bshara · Diyar Husayyan | August 2026

The project in 60 seconds

THE IDEA	A student opens a chat website and writes something like "starting semester 5, failed Data Structures, can't come Sundays" — the agent builds a complete plan: courses, a clash-free weekly timetable, an exam calendar, honest risks. Asking for a change updates the *same* plan instead of starting over.
WHAT MAKES IT SPECIAL	A real agent, not a script: the model decides for itself which tools to call and when a plan is good enough, while Python guarantees the hard rules can never be broken — the model is trusted with judgment, never with correctness.
THE DATA	Three real sources, all fetched once and offline: Technion's own course API, CheeseFork's crowd reviews, and technion-histograms' real grade distributions — cross-checked by hand against each track's official degree diagram. The live site never waits on Technion mid-conversation.
STATUS	**Deployed and live** (not just localhost) across **3 degree tracks**, with persistent cross-session memory, transcript-PDF upload, and agentic grade-improvement retakes. **116 automated checks**, zero API cost, plus a live regression suite run after every model/prompt change. A long live-testing campaign found and fixed roughly two dozen real bugs — see §7.

Try it in 30 seconds

1. **Live:** just open the deployed site — no setup needed.
2. **Locally** — two terminals:

`uvicorn main:app --port 8787 --app-dir agent` + `python3 -m http.server 4173 --directory site`

3 Open the site, pick a track, describe your semester, watch it plan live.

The API key lives in `agent/.env` (gitignored — never committed). Production runs on the shared course LLMod key via Vercel environment variables.

CONTENTS

1 · The story — how we got here

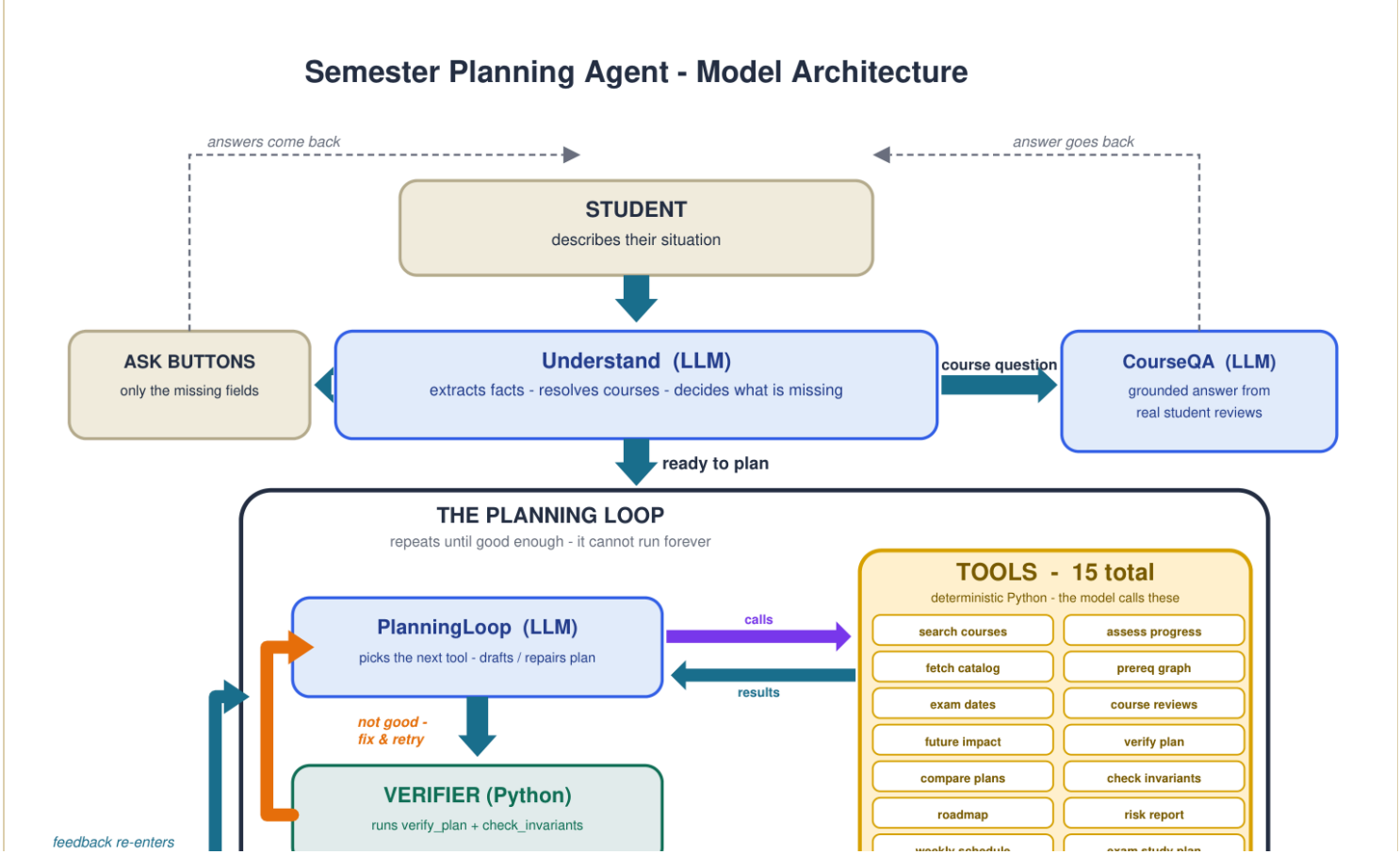
The project went through six real phases, each one changing how the code is shaped. This isn't a tidy plan executed in order — it's what actually happened, including the parts that didn't work the first time.

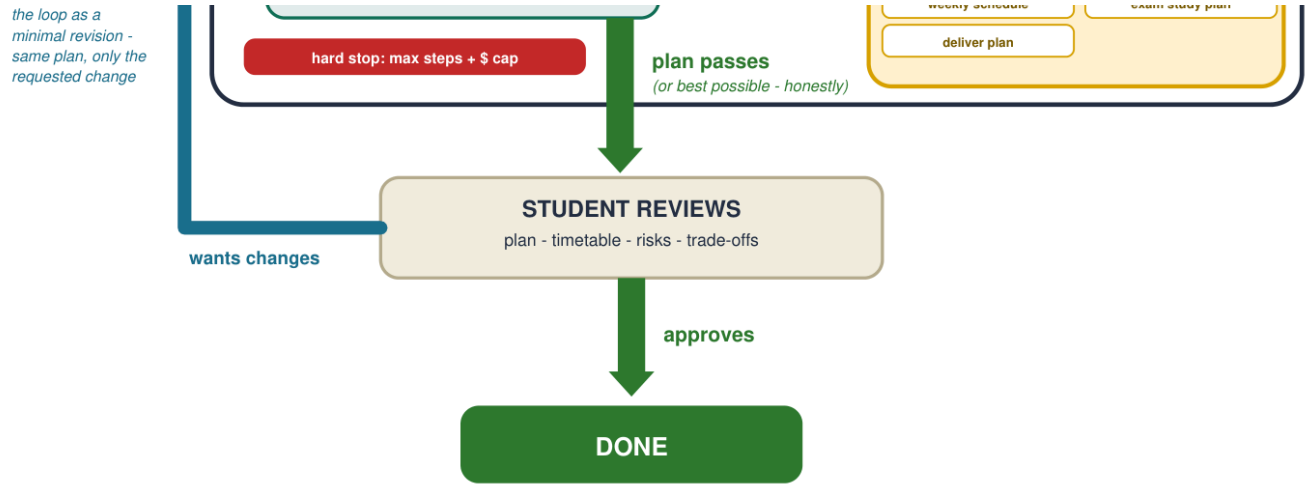
Phase	What happened	What we learned
1. Solve the data	Found Technion's public SAP API and CheeseFork's public review API. Built an offline pipeline packaging everything per track.	The "impossible" data problem had a clean solution hiding in plain sight.
2. First agent attempt	Gave an LLM free rein with tools. It asked permission mid-work, asked for course numbers, rambled.	Never let the model improvise in front of the student.
3. The pipeline version	Rebuilt as a fixed extract → draft-verify-repair → explain script. Reliable, but Python made every decision — a workflow, not an agent.	Reliability comes from bounded loops + a deterministic verifier.
4. The real agent	Rebuilt again: the model chooses which tools to call and when it's done, inside a bounded loop, with every reliability trick kept as a guardrail around it.	Real autonomy and reliability aren't in tension — autonomy just must never touch a student-facing turn unchecked.

5. Feature build-out	Transcript PDF upload, cross-session memory (Supabase), agentic grade-improvement retakes reworked from a fixed threshold into a genuine propose→approve loop, a 3rd degree track.	Building memory first made transcript/grade data worth persisting, rather than each being useful only in isolation.
6. Live-testing hardening	Days of real conversations against the deployed app surfaced structural bugs no unit test would catch — a workload formula that punished normal semesters, a retake guarantee that only lasted one turn, requirement-tree data quietly wrong for specific tracks. Each root-caused via direct reproduction, never guessed, each covered by a new permanent test.	Testing IS development — a live conversation finds the bug a mocked scenario can't imagine to write.

2 · How it works — the big picture

Two loops drive the agent: the inner plan-verify loop keeps improving a candidate until it passes (with hard limits so it can never run forever), and the outer student-feedback loop folds a reply back in as a minimal revision instead of starting over.





Two loops drive the agent: the inner plan-verify loop, and the outer student-feedback loop.
The model is trusted with judgment - never with hard correctness.

Blue = LLM modules. They appear under these exact names in the /api/execute steps trace:
Understand - PlanningLoop - CourseQA

Yellow = the 15 TOOLS (deterministic Python) the PlanningLoop model chooses from at every step.
3 diagnostics auto-run each turn (assess progress, fetch catalog, prereq graph); deliver_plan ends the turn.

Served live at `GET /api/model_architecture` — blue = the three LLM modules (Understand, PlanningLoop, CourseQA), exactly as they appear in the `/api/execute` steps trace; yellow = the 15 deterministic tools; green = verification; red = hard stops; tan = what the student sees.

> The model is trusted with judgment (which courses fit this student) — never with hard correctness. After it delivers, Python re-checks everything and fixes violations before the student sees anything.

3 · A real conversation, step by step

STUDENT

"Starting semester 5, failed Data Structures, can't come Sundays, normal pace."

UNDERSTAND

One LLM call resolves course names (Hebrew/English/typos) to real 8-digit numbers and decides nothing is missing — straight to planning.

PYTHON PREPARES THE GROUND

Computes what a semester-5 student should already have passed, cross-checked against each track's official diagram data, and builds the real shortlist of what's actually takeable next semester.

THE AGENT PLANS

Drafts around the Data Structures retake (rule #1: retakes come first), checks exam dates and reviews, verifies, fixes an exam clash, verifies again, checks risk, delivers.

STUDENT SEES

An animated plan: stat tiles, course cards each with a "why," a weekly timetable, exam calendar, top risk, and an honest note on anything unresolved.

STUDENT

"Swap the statistics course, keep everything else."

AGENT

Updates the *same* plan — one course out, a replacement in, everything else untouched — and the site shows exactly what changed (kept / removed / added).

Grade-improvement retake, short scenario

STUDENT

"...I got a 65 in Linear Algebra."

AGENT

[delivers the plan] "...one more thing — your grade in Linear Algebra (65) is well below both the course average and your own average. Want me to add a retake next time?"

STUDENT

"yes"

AGENT

Next plan includes Linear Algebra, tagged **RETAKE** — and it stays in the plan through every later revision this conversation, not just this one turn.

4 · The data — where it comes from, what we did to it

Source	What we take	Access
Technion's own course system (SAP)	Every course: name, credit points, prerequisites (AND/OR trees), weekly schedule with all class sections, exam dates (moed A+B), degree requirement tree per track	Public API, no password
Official DDS ("track diagram") PDFs	Ground-truth per-course semester placement, hand-digitized and cross-checked against each diagram's own printed credit totals — overrides the requirement tree wherever the tree is wrong or incomplete	Official Technion PDF, digitized by hand

	of incomplete	
CheeseFork	Crowd difficulty/satisfaction ratings (1–5) and real review excerpts, quoted directly	Public API, no password
technion-histograms	Real historical grade distributions per course — the comparison baseline behind grade-improvement retakes	Public dataset

What the pipeline does (runs once, offline)

- Walks each track's degree requirement tree and collects every course it mentions (137–149 per track)
- Parses raw prerequisite data into proper AND/OR logic trees
- Precomputes the reverse-prerequisite graph ("what does each course unlock") — powers retake priority, risk reporting, and the graduation roadmap at zero runtime cost
- Attaches CheeseFork review data and technion-histograms grade averages to each course
- Writes one JSON bundle per track; the server loads them at startup — requests never wait on Technion, except one narrow real-time "is this still offered" check right before delivery

The tricky data problems we found and fixed

- **No official "semester N" label exists** for most courses — inferred from prerequisite-chain depth, then overridden per-course wherever the real diagram data is known.
- **Technion's requirement tree is shared across tracks/faculties** — it sometimes pairs a track's real, already-required course variant with a completely different track's variant of the same subject under one fake "choose one" node. Every such "choice group" we found turned out to have exactly one real option and one phantom — never a genuine open choice once diagram data exists.
- **The tree is also incomplete** — missing real mandatory courses the diagrams show, in the other direction from the phantom-choice problem above.
- **"Meets on Sunday" is often true for only one section** of a course — checked per-section, which removed roughly half of false schedule conflicts.
- **Course numbers appear with and without leading zeros** — normalized to 8 digits everywhere.

5 · The agent, explained simply

"Agent" has a precise meaning here: the AI itself decides what to do next — which check to run, whether the plan is good enough, when to stop — the way a person uses a calculator and a checklist, not a script following fixed steps. Verified live: identical-looking requests produce genuinely different, self-chosen tool sequences and step counts.

Its toolbox — 15 tools, all plain Python (no AI inside them)

Group	Tools	Used for

Check the plan	verify_plan, check_invariants	The critic: prerequisites, exam spacing, credits, workload, schedule conflicts, data integrity
Understand the degree	fetch_catalog, query_prereq_graph, assess_progress, roadmap_to_graduation, simulate_future_impact	What's left, what blocks what, is the student behind, what today's choices unlock later
Judge quality	summarize_cheesefork, fetch_exam_dates, risk_report, compare_plans	Are these courses liked, do exams collide, what could go wrong, which of two candidates is better
Live with the plan	weekly_schedule_analyzer, exam_study_planner	What the week really looks like; whether the exam period leaves real study days (moed A + B)
Find + finish	search_courses, deliver_plan	Resolve a course name; hand over the final plan — the model's own "I'm done" decision

The safety rules (Python pushes back, once each, then enforces)

- A plan must be verified before delivery
- No half-empty semesters; no padding with more than one sport/PE course
- A student's specific complaint may not survive into the "fixed" plan
- "Remove course X" — X can never appear in the delivered plan, period, enforced in code on the second violation
- A delivered week can never contain a class-time collision or a course with unmet prerequisites — both stripped in code with an honest note
- An explicit "add course X" request either lands in the plan, or is explained by name and — if only a schedule/room trade-off, never if it's simply not offered — negotiated with a direct question and honored on confirmation
- A confirmed grade-improvement retake persists for the rest of the conversation, not just the turn it was approved — an unrelated later revision can never silently drop it
- Hard limits every turn: a max tool-call count and a dollar cap — it can never loop forever

6 · Everything the app can do

Talking to it	Natural free text, Hebrew or English — buttons appear only for genuinely missing info — instant grounded answers to course questions — the working card shows the agent's real steps live while it thinks

	the agent's real steps live while it thinks
Onboarding	Manual course checklist, **or upload an official Technion transcript PDF** — parsed directly into passed/failed courses and grades, never stored beyond the request unless memory is enabled
The plan you get	Animated reveal, counting stat tiles, course cards with a "why," weekly timetable, exam calendar, top risk + graduation roadmap, honest notes on anything unresolved, confetti on a fully verified plan
Changing it	Say it, tap a quick-reply chip, or drag a course off the timetable — the same plan updates minimally with a kept/removed/added diff shown
Grade-improvement retakes	Agentic propose→approve (the agent asks, never assumes) or student-initiated — hard-enforced across every delivery path and persisted for the whole conversation
Cross-session memory	Optional, keyed by a hashed `@campus.technion.ac.il` email — restore a saved plan on a different browser/device, no password, matching the course spec's no-login-wall requirement
Back to previous plan	Restores a prior plan in full (not just its course list) — Print/Add to calendar/Share image all keep working on it, and the next message revises from the restored plan
Extras	.ics calendar export, print sheet, shareable PNG card, full Hebrew RTL mode, optional soft sounds, **3 degree tracks**

7 · How we know it works

- **116 automated checks**, zero API cost (`bash agent/tests/run_mocked.sh`) — simulate the model and lock in every guarantee: facts never forgotten between turns, a revision never returns a stranger plan, an explicit directive is never silently undone, guards fire exactly once, a confirmed retake survives an unrelated later turn.
- A live regression suite — synthetic students run against the real model after every model/prompt change, most recently re-verified against the shared course LLMod key after switching back to it for submission.
- A leveled test plan (`TESTING.md`) — smoke tests first, then basics, core features, advanced behavior, then hard/adversarial cases, with a gate between levels.

Real bugs found live, root-caused, fixed — a representative sample

What we caught
A workload formula scored a completely ordinary mandatory-heavy semester as "too heavy," so the model

Technion's shared requirement tree paired a track's real course requirement with a phantom variant from a

A confirmed grade-improvement retake was only ever protected on the exact turn it was approved — Pytho

"Do you recommend retaking X?" couldn't see the student's own already-known grade — a different code p

An explicit "add course X" request could get force-added, then silently re-stripped by a later check with ze

The elective menu was effectively static — the same one or two courses suggested every time for a given s

A hung network call could freeze a turn for minutes; OpenAI rate-limits crashed turns outright

"Sundays are fine now" (relaxing a constraint) was silently ignored — the state merge couldn't distinguish

Delivered weeks could contain overlapping class times, never checked anywhere

A prerequisite-locked course could be knowingly included as an "anchor"

Every fix above shipped with a new permanent regression test, verified live against the deployed app, not just locally.

8 · Team handbook

Where everything lives

Path	What it is
agent/agent_react_loop.py	**THE AGENT** — the bounded tool-choosing loop and every guard
agent/tools.py	15 model-callable tools (pure Python, no AI inside) plus grade-improvement analysis
agent/agent_loop.py	Understanding/extraction + shared helpers + the legacy pipeline (fallback)
agent/student_store.py	Supabase-backed session persistence, keyed by hashed student email
agent/transcript_parser.py	Parses an uploaded Technion transcript PDF into courses/grades
agent/live_offering_check.py	Real-time "is this course still offered" check right before delivery
agent/data_bundle.py	Loads `data/track_*.json`, official per-course semester data, choice-group logic
agent/main.py	Web server: `/chat`, `/chat/stream`, `/transcript`, `/session`, `/tracks`, `/health`, plus the course-spec-required `/api/*` endpoints
agent/tests/	Automated checks + live scenario suites
site/index.html	The whole website — one file, no build step
pipeline/	Offline data fetching (Technion SAP API + CheeseFork + technion-histograms)
data/track_*.json	Packaged per-track course bundles (3 tracks) the server loads

Deployment and access

- Runs on Vercel (serverless), deployed from the personal GitHub repo used for this project's submission.
- Production uses the shared course LLMod key (`OPENAI_BASE_URL=https://api.llmod.ai/v1`, model `MB5R2CF-`

`azure/gpt-5.4-mini`) as Vercel environment variables — verified working with a fresh live regression pass after every key/config change.

- Entry point for Vercel: `api/index.py` + `vercel.json`.

Honest limitations (so nobody is surprised)

> - Technion's real grade-improvement policy (שיפור ציון) — exact eligibility, attempt limits, which grade counts — is not modeled; every retake proposal is framed as worth confirming with the registrar, never guaranteed.
> - One track's semester-5+ "specialization chain" requirement isn't modeled yet; semesters 1–4 are fully accurate for all 3 tracks.
> - Calendar export approximates the semester's start date; exam dates are exact.
> - The roadmap is heuristic layering over real prerequisite data, not an official Technion sequence.
> - Model variance: the same prompt can yield different, equally valid plans — judged against the acceptance contract in `TESTING.md`, not a remembered plan.
